

TP : Les Middlewares

Partie 1 : Création et Utilisation d'un Middleware Personnalisé

1.1 Génération du Middleware

Expliquez aux étudiants qu'un middleware est une classe PHP qui se trouve entre la requête HTTP et la logique de l'application.

- Générez un nouveau middleware pour vérifier si l'heure est autorisée (par exemple, bloquer l'accès après 18h).

```
php artisan make:middleware CheckWorkingHours
```

La classe sera créée dans **app/Http/Middleware/CheckWorkingHours.php**.

1.2 Implémentation de la Logique

Dans la méthode **handle** du middleware **CheckWorkingHours**, ajoutez la logique :

```
// app/Http/Middleware/CheckWorkingHours.php

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class CheckWorkingHours
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request):
     * (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        // vérifie si l'heure actuelle est supérieure ou égale à 18h (6 PM)
        if (now()->hour >= 18) {
            // Renvoie une réponse d'erreur 403 (Interdit)
            return response('Désolé, cette page est accessible uniquement
avant 18h.', 403);
        }

        // Si l'heure est correcte, passe la requête au prochain élément
        (route/middleware)
        return $next($request);
    }
}
```

```
}
```

1.3 Enregistrement et Alias du Middleware

Depuis Laravel 11, l'enregistrement se fait dans le fichier **bootstrap/app.php**. Enregistrez le middleware en tant qu'alias pour une utilisation facile dans les routes :

- Ouvrez **bootstrap/app.php**.
- Localisez la fonction **withMiddleware** et ajoutez un alias :

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware): void {
    // ... autres configurations de middleware ...

    $middleware->alias([
        // votre alias de middleware
        'work.time' => \App\Http\Middleware\CheckworkingHours::class,
    ]);
});
```

1.4 Application à une Route

- Créez une route dans **routes/web.php** et appliquez le middleware :

```
// routes/web.php

use Illuminate\Support\Facades\Route;

// Route accessible uniquement pendant les heures de travail
Route::get('/special-access', function () {
    return 'Bienvenue! Il est encore l\'heure de travailler.';
})->middleware('work.time'); // utilisation de l'alias
```

- **Test** : Demandez aux étudiants de tester la route **avant 18h** (doit fonctionner) et de simuler le passage de l'heure (en modifiant la condition dans le middleware) pour voir le **blocage 403**.

Partie 2 : Le Middleware CSRF

2.1 Compréhension du CSRF

Expliquez la vulnérabilité **Cross-Site Request Forgery** (CSRF) et comment le middleware **\Illuminate\Foundation\Http\Middleware\VerifyCsrfToken** protège l'application en vérifiant un jeton secret.

2.2 Création d'un Formulaire POST sans Protection

- Créez une route GET pour afficher un formulaire :

```
// routes/web.php

Route::get('/form', function () {
    return view('csrf-form');
});
```

```
});
```

- Créez une route POST pour traiter les données :

```
// routes/web.php
Route::post('/submit-data', function (Request $request) {
    // Note: Ajouter 'use Illuminate\Http\Request;' au début du fichier
    return "Données soumises (aucune validation CSRF ici) : " . $request->input('data');
});
```

- Créez une vue **resources/views/csrf-form.blade.php** SANS le jeton CSRF :

```
<!DOCTYPE html>
<html>
<head><title>Formulaire sans CSRF</title></head>
<body>
    <h1>Soumettre des données (sans protection CSRF)</h1>
    <form method="POST" action="/submit-data">
        <input type="text" name="data" placeholder="Entrez une donnée">
        <button type="submit">Soumettre</button>
    </form>
</body>
</html>
```

- **Test** : Soumettez le formulaire. Les étudiants devraient voir une erreur **419 PAGE EXPIRED** (ou **TokenMismatchException** dans les anciennes versions), car le middleware **CSRF** (qui fait partie du groupe de middlewares web appliqué par défaut) a rejeté la requête.

2.3 Application de la Protection CSRF

- Modifiez la vue **csrf-form.blade.php** pour inclure le jeton :

```
...
<form method="POST" action="/submit-data">
    @csrf
    <input type="text" name="data" placeholder="Entrez une donnée">
    <button type="submit">Soumettre</button>
</form>
...
```

- **Test** : Soumettez à nouveau. La soumission doit fonctionner (affichant le message de la route POST), car le jeton CSRF est maintenant inclus et validé par le middleware.